integrated ethics LABS

Hospitality in Computer Programming

Lori Carter

2021-07-21

Student Materials

Worksheet

- web
- PDF

Acknowledgement

This lab was motivated, in part, by a paper written by Victor Norman of Calvin University. https://cs.calvin.edu/static/documents/christian/TeachingHospitableCode.pdf

Overview

Ethics background desired: Virtue Ethics, Analogies, Consequentialism, & Deontology. If students or professor are not familiar with these frameworks, a brief summary is provided in the reflection.

Subject matter referred to in this lab: Hospitable computer programming (Exercise written in C++ (uses pointers and vectors), and also Java). Could be adapted to another language if preferable.

Placement in overall ethics curriculum:

- Academic year: Years 2-4. To reinforce the need for good programming techniques we recommend using it earlier rather than later! We use this first semester of sophomore year.
- Recommended previous labs: Foundational Labs
- Recommended follow-up labs: Reliability Lab

Time required:

- Out of class: None (but potential online version available if unable to do in class)
- In class: 25-30 mins

Learning objectives:

- Consider the importance of documentation, good programming practices, and user-friendliness from perspectives of both users and developers/programers.
- Practice using an ethical framework to discuss the need for hospitable coding with a team member.

Ethical dilemma or issue to be considered: Hospitality

Hospitality is defined by Dictionary.com as

> the friendly reception and treatment of guests or strangers.

As students, often the only motivation for hospitable programming practices is the grade. This exercise should help point out that it is frustrating and time-consuming to both code users and code maintainers (our guests and strangers) to try to deal with code that is not well-written and well-documented.

Flow

- Explore the code
  - Run the code given, asking the class to figure out what the game is from only the output, OR
  - Proved the code electronically and have students run it themselves. (Using this option, some of this activity could become an assignment to be done outside of class.)
- Task students with making it more user-friendly. (Provide printed or electronic version of code.)
- Discuss the challenges encountered in altering the code.
- Reflect on past and future behavior with regard to hospitable coding.

Preparation

- Read the entire lab.
- Decide which language you would like to present the exercise in (C++ and Java versions are provided but you may want to transcribe to Python or another language to meet your needs).
- Load the chosen code on your computer and make sure that it runs.
- Print copies of the code, or make the code available electronically, so that each student or pair of students can have a copy.
- Create a way to display (PowerPoint slide or other) the reasons that students may not use hospitable programming practices and possibly review the ethical frameworks.

Lesson plan

Introduction (5 min)

Start by running (or having students run) the code. If running it as a demo, choose a student to play the coded game and tell the class that they are tasked with figuring out what the game does as their peer plays it. The prompts for the user are one of the inhospitable aspects of the code so let the student chosen to interact with the game struggle in responding. (The game is similar to Blackjack, but the goal is 20 instead of 21. This may also be an inhospitable aspect since there are no instructions). For the C++ version, a "seed" is required for the random number generator and this is also awkward.

Activity (15 min)

- Ask students (working alone or in groups to 2) to work on making the code more user friendly. (In order to do this, they will have to try to understand the code, and this presents many other inhospitable aspects of the code.)
- When the class comes back together, ask students what they changed in order to make the code more user-friendly. Hopefully, some of them can talk about adding instructions, adding better prompts etc.
- Whether the students were successful or not, talk about what made the process difficult. Answers should include:
  - No comments
  - Meaningless variable names
  - The use of pointers in the C++ code
  - Meaningless function names

Reflection (10 min)

Ask students to consider (silently) common student behaviors with respect to hospitable programming practices. They should be able to relate to many! This list should be displayed as previously prepared.

- I put in plenty of comments after my lab is done, so I don't have to worry about them while I'm working.
- I only put in comments so that I don't get marked down a bunch of points.
- I don't really understand what my professor means when they say that they want good comments, so I just guess or throw in a bunch of random stuff hoping it works.

- I comment my code before I actually work on it because they give me an outline for what I'm doing.
- I don't need to put comments on repetitious code because the first comments already explain what the rest of it is doing.
- The code I wrote is so easy to understand that anyone would get it, so I don't really need any comments.
- In the end, what really matters is the fact that my program works, so I don't see the point of putting in comments or making meaningful variable names.
- My comments are a cry of help to whoever is grading my labs. (Ex. "I thought this section was working, but now it's not. Please give me credit :,(")
- I put comments above each method and call it good. This relieves me from making a cumbersome method name and I don't need anything on the inside of the method.
- I don't put a comment saying that I wrote the file because I don't need to, since I'm the only one using it.
- I comment my code so that I naturally do it when I have a job and other people are looking at my work.
- I comment my code so that I can quickly figure out what is going on when I want to update my code.

Discuss

- Did any of the student programming behaviors resonate with you? How?
- How might you think differently about comments, variable names, and user prompts based on this exercise?
- What are some of the hallmarks of good comments?

Consider this. You are the manager of a software team. Some of your team members are writing code without good variable names or comments. One way to address the situation would be with a deontological approach. Recall that the deontological ethical framework relies on a set of rules. One such rule may be that "variable names must be meaningful and you just use comments at least every 5 lines in your code." How do you think this would go over with the team member? (Probably be resistant at being commanded to do something without a good reason)

Would there be a better way to approach the team member? Suggest a method for approaching the team member using one of the other ethical frameworks:

- Virtue ethics: making a decision based on whether the outcome upholds a set of virtues (honesty, compassion, patience, hard-work, kindness are some examples)

- Analogies: making a decision based on a similar situation where the ethicality of the similar situation was widely accepted
- Consequentialism: making a decision based on the consequences and weighing harms and benefits. ### Assessment (to be included on a later quiz or exam)
- What are some inhospitable practices when it comes to computer programming? What is inhospitable about them?
  - poor user instructions, not indicating who wrote the code, meaningless variable names, less readable code (pointers, “clever” algorithms), no comments
- How would you help someone come to see that inhospitable programming practices are actually an ethical issue? Use one of the 4 ethical frameworks (virtue ethics, deontology, analogies, consequentialism) in your answer.

Example Code

Select one of the languages blow.

C++ Java Python R

```{C, eval = FALSE} #include #include #include using namespace std;

bool w(vector y) { int z = 0; for (int i = 0; i < y.size(); i++) z += y.at(i); if(z>20) return false; return true; } void f(int * r, bool a) { if (r == 14 && a) r = 11; else if (r == 14 && !a) r = 1; else if (r > 10) r = 10; } int main() { int seed; cin >> seed; srand(seed); char n,c; do{ vector x; int p = 0; int * h = &p; p = rand() % 14 + 1; f(h,true); x.push_back(p); int t=0; do{ p = rand() % 14 + 1; if (t <= 10) f(h, true); else f(h, false); x.push_back(p); //cout << “you have” << endl; t = 0; for (int i = 0; i < x.size(); i++) { t += x.at(i); cout << x.at(i) << ” “; } cout << endl; if (w(x)) { cout <<”Another?” << endl; cin >> c; } } while (w(x) && c==‘y’); int o = rand() % 20; if (w(x) && o < t) cout << “You win! :)” << endl; else if (w(x) && o > t) { cout << “You got beat :(“ << endl; cout << “Other person had” << o << endl; } else if (w(x) && o == t) cout << “You tied :/” << endl; else cout << “You lose. :(“ << endl; cout << “again? y or n” << endl; cin >> n; } while (n == ‘y’); cin.ignore(1); return 0; }

```

```{java, eval = FALSE}
import java.util.*;

public class hMod
{
   public static boolean w(ArrayList<Integer> y)
   {
      int z = 0;
      for (int i = 0; i < y.size(); i++)
```

```
        z += y.get(i);
        if(z>20)
           return false;
     return true;
  }
  public static void f(int r, boolean a)
  {
     if (r == 14 && a)
        r = 11;
     else if (r == 14 && !a)
        r = 1;
     else if (r > 10)
        r = 10;
  }
  public static void main(String[]args)
  {
     Scanner sc=new Scanner(System.in);
     Random srand = new Random();
     char n,c='y';
     do{
        ArrayList <Integer> x = new ArrayList<Integer>();
        int p = 0;

        p = srand.nextInt(14) + 1;
        System.out.println("first p "+p);
        f(p,true);
        x.add(p);
        int t=0;
        do{
           p = srand.nextInt(14) +1;
           System.out.println("next p "+p);
           if (t <= 10)
              f(p, true);
           else
              f(p, false);
           x.add(p);
           //System.out.println("you have " );;
           t = 0;
           for (int i = 0; i < x.size(); i++) {
              t += x.get(i);
              System.out.print( x.get(i) + " ");
           }
           System.out.println();
           if (w(x)) {
              System.out.println("Another?" );
              c =sc.next().charAt(0);
           }
```

```
         } while (w(x) && c=='y');
         int o = srand.nextInt(20);
         if (w(x) && o < t)
         {
            System.out.println( "You win! :)");
            System.out.println("Other person had " + o);}
         else if (w(x) && o > t) {
            System.out.println("You got beat :(" );
            System.out.println("Other person had " + o);
         }
         else if (w(x) && o == t)
            System.out.println( "You tied :/");
         else
         {
            System.out.println("You lose. :(");
            System.out.println("Other person had " + o);
            }
         System.out.println("again? y or n" );
         n=sc.next().charAt(0);
      } while (n == 'y');

   }
}
```

Python code coming soon.

R code coming soon.

Bibliography